Detail Assignment

SOAL 1 — Data Loading & Profiling Awal

Perintah 1:

Load keempat CSV menjadi DataFrame Spark dengan schema yang benar (pastikan order_date dan signup_date bertipe date, total_amount, price double, quantity int).

Perintah 2:

Lakukan data profiling ringkas:

1. jumlah baris tiap DataFrame,
2. cek null per kolom,
3. tampilkan 5 contoh baris tiap dataset.

Perintah 3:

Buat “anomaly summary” awal:

1. jumlah orders dengan total_amount <= 0,
2. jumlah order_items dengan quantity <= 0 atau price <= 0,
3. jumlah duplikasi key utama (order_id di orders, product_id di products, user_id di users).

SOAL 2 — Cleaning & Standardization

Perintah 1:

Bersihkan data:

1. drop baris orders yang order_id atau user_id null,
2. drop baris order_items yang order_id atau product_id null,
3. drop baris products yang product_id null,
4. drop baris users yang user_id null.

Perintah 2:

Standardisasi kolom teks:

1. city di users → lowercase + trim,
2. category di products → lowercase + trim.

Perintah 3:

Deduplication berbasis key:

1. orders dedupe by order_id,
2. order_items dedupe by kombinasi (order_id,product_id),
3. products dedupe by product_id,
4. users dedupe by user_id. Tunjukkan jumlah baris sebelum vs sesudah dedupe.

SOAL 3 — Data Enrichment & Feature Engineering

Perintah 1:

Di order_items, buat kolom turunan:

1. total_item_value = quantity * price.

Perintah 2:

Di orders, buat kolom kategorisasi menggunakan when/otherwise:

1. order_value_flag dengan aturan:
   high_value jika total_amount > 500000, selain itu normal.

Perintah 3:

Buat kolom segmentasi user di users:

1. user_tenure_days = datediff(order_date_max_per_user, signup_date) (Hint: butuh join/agg kecil untuk order_date_max_per_user).
   tenure_segment = new (<=30 hari), mature (31–180), loyal (>180).

SOAL 4 — Join & Aggregation (Business Metrics)

Perintah 1:

Bangun DataFrame “fact” hasil join:
orders ↔ order_items ↔ products ↔ users
dengan key:

1. orders ↔ order_items: order_id
2. order_items ↔ products: product_id
3. orders ↔ users: user_id

Perintah 2:

Hitung metrik per **category per day**:

1. daily_gmv_category = sum(total_item_value)
2. daily_orders_category = countDistinct(order_id)
3. daily_items_sold = sum(quantity)
   Urutkan per hari dan GMV terbesar.

Perintah 3:

Hitung metrik per **city**:

1. total GMV kota,
2. AOV kota (avg total_amount),
3. jumlah customer unik kota,
4. top 3 kategori kontribusi terbesar per kota (boleh pakai window rank).

SOAL 5 — Window Analytics & Insight Final

Perintah 1:

Gunakan window function per user untuk membuat kolom:

1. user_total_spending (sum total_amount per user),
2. user_total_orders (count order per user),
3. user_avg_order_value (avg total_amount per user),
4. user_rank global berdasarkan user_total_spending (dense_rank).

Perintah 2:

Buat analisis **cumulative GMV harian**:

1. gmv_daily,
2. cumulative_gmv (sum gmv_daily over order by date).

Perintah 3:

Ambil **Top 10 customers** (rank 1–10) dan tampilkan:

user_id, name, city, user_total_orders, user_total_spending, user_avg_order_value. Tambahkan interpretasi singkat 4–6 kalimat: pola apa yang kamu lihat? (misal kota dominan, AOV tinggi tapi order sedikit, dll).

Pengumpulan Assignment

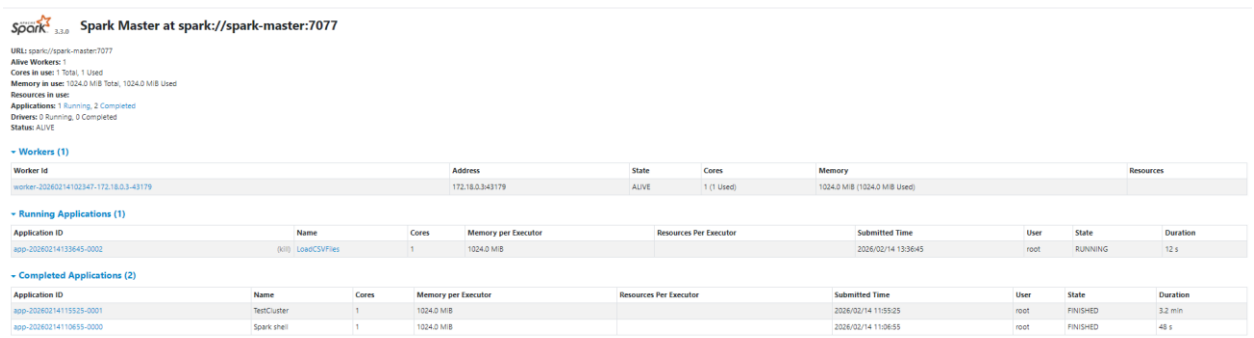
Assignment

SOAL 1 — Data Loading & Profiling Awal

Perintah 1:

Load keempat CSV menjadi DataFrame Spark

a. pastikan order_date dan signup_date bertipe date,
b. total_amount, price double,
c. quantity int




```
26/02/14 13:46:00 INFO DAGScheduler: Job 7 finished: csv at NativeMethodAccessorImpl.java:0, took 0.431730 s
root
 |-- user_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- city: string (nullable = true)
 |-- signup_date: date (nullable = true)

root
 |-- product_id: integer (nullable = true)
 |-- user_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- city: string (nullable = true)
 |-- signup_date: date (nullable = true)

root
 |-- product_id: integer (nullable = true)
 |-- product_id: integer (nullable = true)
 |-- product_name: string (nullable = true)
 |-- category: string (nullable = true)
 |-- price: double (nullable = true)

root
 |-- order_id: integer (nullable = true)
 |-- category: string (nullable = true)
 |-- price: double (nullable = true)

root
 |-- order_id: integer (nullable = true)
 |-- price: double (nullable = true)

root
 |-- order_id: integer (nullable = true)
root
 |-- order_id: integer (nullable = true)
 |-- user_id: integer (nullable = true)
 |-- order_date: date (nullable = true)
 |-- total_amount: double (nullable = true)

 |-- order_id: integer (nullable = true)
 |-- user_id: integer (nullable = true)
 |-- order_date: date (nullable = true)
 |-- total_amount: double (nullable = true)

 |-- user_id: integer (nullable = true)
 |-- order_date: date (nullable = true)
 |-- total_amount: double (nullable = true)
```

```
root
 |-- order_id: integer (nullable = true)
 |-- user_id: integer (nullable = true)
 |-- order_date: date (nullable = true)
 |-- total_amount: double (nullable = true)

 |-- order_id: integer (nullable = true)
 |-- user_id: integer (nullable = true)
 |-- order_date: date (nullable = true)
 |-- total_amount: double (nullable = true)

 |-- user_id: integer (nullable = true)
 |-- order_date: date (nullable = true)
 |-- total_amount: double (nullable = true)

root
 |-- order_id: integer (nullable = true)
 |-- product_id: integer (nullable = true)
root
 |-- order_id: integer (nullable = true)
 |-- product_id: integer (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- price: double (nullable = true)

 |-- quantity: integer (nullable = true)
 |-- price: double (nullable = true)
```

Source code

```python
spark = SparkSession.builder \
    .appName("LoadAndCastCSV") \
    .master("spark://spark-master:7077") \
    .getOrCreate()

base_path = "/spark-scripts/assignment_Working with PySpark DataFrames for Analytics/"

# Load CSV
users_df = spark.read.csv(base_path + "users.csv", header=True, inferSchema=True)
products_df = spark.read.csv(base_path + "products.csv", header=True, inferSchema=True)
orders_df = spark.read.csv(base_path + "orders.csv", header=True, inferSchema=True)
order_items_df = spark.read.csv(base_path + "order_items.csv", header=True, inferSchema=True)

# ◆ Fix tipe data

users_df = users_df.withColumn(
    "signup_date",
    to_date(col("signup_date"))
)

products_df = products_df.withColumn(
    "price",
    col("price").cast("double")
)

orders_df = orders_df.withColumn(
    "order_date",
    to_date(col("order_date"))
).withColumn(
    "total_amount",
    col("total_amount").cast("double")
)

order_items_df = order_items_df \
    .withColumn("quantity", col("quantity").cast("int")) \
    .withColumn("price", col("price").cast("double"))

# Cek hasil schema
users_df.printSchema()
products_df.printSchema()
orders_df.printSchema()
order_items_df.printSchema()

spark.stop()
```

Perintah 2:

Lakukan data profiling ringkas:

1. jumlah baris tiap DataFrame,
2. cek null per kolom,
3. tampilkan 5 contoh baris tiap dataset.

```
def data_profiling(df, df_name):
    print(f"\n===== DATA PROFILING: {df_name} =====")
    total_rows = df.count()
    print(f"Total Rows: {total_rows}")
    print("\nNull Count per Column:")
    df.select([
        count(when(col(c).isNull(), c)).alias(c)
        for c in df.columns
    ]).show()
    print("\nSample Data:")
    df.show(5)

# Jalankan profiling untuk semua dataset
data_profiling(users_df, "USERS")
data_profiling(products_df, "PRODUCTS")
data_profiling(orders_df, "ORDERS")
data_profiling(order_items_df, "ORDER_ITEMS")
```

```
===== DATA PROFILING: USERS =====
26/02/14 14:01:23 INFO FileSourceStrategy: Pushed Filters:
26/02/14 14:01:23 INFO FileSourceStrategy: Post-Scan Filters:
26/02/14 14:01:23 INFO FileSourceStrategy: Output Data Schema: struct<>
26/02/14 14:01:23 INFO CodeGenerator: Code generated in 19.559663 ms
26/02/14 14:01:23 INFO MemoryStore: Block broadcast_16 stored as values in memory (estimated size 199.2 KiB, free 433.9 MiB)
26/02/14 14:01:23 INFO BlockManagerInfo: Removed broadcast_14_piece0 on spark-master:32891 in memory (size: 34.0 KiB, free: 434.4 MiB)
26/02/14 14:01:23 INFO BlockManagerInfo: Removed broadcast_14_piece0 on 172.18.0.3:35307 in memory (size: 34.0 KiB, free: 434.4 MiB)
26/02/14 14:01:23 INFO MemoryStore: Block broadcast_16_piece0 stored as bytes in memory (estimated size 33.9 KiB, free 434.1 MiB)
26/02/14 14:01:23 INFO BlockManagerInfo: Added broadcast_16_piece0 in memory on spark-master:32891 (size: 33.9 KiB, free: 434.4 MiB)
26/02/14 14:01:23 INFO SparkContext: Created broadcast 16 from count at NativeMethodAccessorImpl.java:0
```

```
26/02/14 14:01:24 INFO BlockManagerInfo: Removed broadcast_17_piece0 on 172.18.0.3:35307 in memory (size: 8.2 KiB, free: 434
26/02/14 14:01:24 INFO TaskSetManager: Starting task 0.0 in stage 10.0 (TID 9) (172.18.0.3, executor 0, partition 0, NODE_LO
26/02/14 14:01:24 INFO BlockManagerInfo: Added broadcast_18_piece0 in memory on 172.18.0.3:35307 (size: 5.5 KiB, free: 434.4
26/02/14 14:01:24 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 0 to 172.18.0.3:46918
26/02/14 14:01:25 INFO TaskSetManager: Finished task 0.0 in stage 10.0 (TID 9) in 584 ms on 172.18.0.3 (executor 0) (1/1)
26/02/14 14:01:25 INFO TaskSchedulerImpl: Removed TaskSet 10.0, whose tasks have all completed, from pool
26/02/14 14:01:25 INFO DAGScheduler: ResultStage 10 (count at NativeMethodAccessorImpl.java:0) finished in 0.630 s
26/02/14 14:01:25 INFO DAGScheduler: Job 9 is finished. Cancelling potential speculative or zombie tasks for this job
26/02/14 14:01:25 INFO TaskSchedulerImpl: Killing all running tasks in stage 10: Stage finished
26/02/14 14:01:25 INFO DAGScheduler: Job 9 finished: count at NativeMethodAccessorImpl.java:0, took 0.654770 s
Total Rows: 500
```

```
26/02/14 14:01:26 INFO CodeGenerator: Code generated in 17.22138 ms
+-------+----+----+-----------+
|user_id|name|city|signup_date|
+-------+----+----+-----------+
|      0|   0|   0|          0|
+-------+----+----+-----------+
```

```
26/02/14 14:01:26 INFO SparkContext: Created broadcast 23 from broadcast at DAGScheduler.scala:1513
26/02/14 14:01:26 INFO DAGScheduler: Submitting 1 missing tasks from ResultStage 14 (MapPartitionsRDD[57] at showString
26/02/14 14:01:26 INFO TaskSchedulerImpl: Adding task set 14.0 with 1 tasks resource profile 0
26/02/14 14:01:26 INFO TaskSetManager: Starting task 0.0 in stage 14.0 (TID 12) (172.18.0.3, executor 0, partition 0, P
26/02/14 14:01:26 INFO BlockManagerInfo: Added broadcast_23_piece0 in memory on 172.18.0.3:35307 (size: 7.7 KiB, free:
26/02/14 14:01:26 INFO BlockManagerInfo: Added broadcast_22_piece0 in memory on 172.18.0.3:35307 (size: 33.9 KiB, free:
26/02/14 14:01:26 INFO TaskSetManager: Finished task 0.0 in stage 14.0 (TID 12) in 198 ms on 172.18.0.3 (executor 0) (1
26/02/14 14:01:26 INFO TaskSchedulerImpl: Removed TaskSet 14.0, whose tasks have all completed, from pool
26/02/14 14:01:26 INFO DAGScheduler: ResultStage 14 (showString at NativeMethodAccessorImpl.java:0) finished in 0.225 s
26/02/14 14:01:26 INFO DAGScheduler: Job 12 is finished. Cancelling potential speculative or zombie tasks for this job
26/02/14 14:01:26 INFO TaskSchedulerImpl: Killing all running tasks in stage 14: Stage finished
26/02/14 14:01:26 INFO DAGScheduler: Job 12 finished: showString at NativeMethodAccessorImpl.java:0, took 0.244134 s
26/02/14 14:01:26 INFO CodeGenerator: Code generated in 17.3441 ms
+-------+--------+--------+-----------+
|user_id|    name|    city|signup_date|
+-------+--------+--------+-----------+
|    101|User_101|   Depok| 2022-01-01|
|    102|User_102|   Medan| 2022-01-02|
|    103|User_103|   Depok| 2022-01-03|
|    104|User_104|Surabaya| 2022-01-04|
|    105|User_105|   Depok| 2022-01-05|
+-------+--------+--------+-----------+
only showing top 5 rows
```

```
===== DATA PROFILING: PRODUCTS =====
26/02/14 14:01:26 INFO FileSourceStrategy: Pushed Filters:
26/02/14 14:01:26 INFO FileSourceStrategy: Post-Scan Filters:
26/02/14 14:01:26 INFO FileSourceStrategy: Output Data Schema: struct<>
26/02/14 14:01:26 INFO MemoryStore: Block broadcast_24 stored as values in memory (estimated size 199.2 KiB, free 434.0 MiB)
26/02/14 14:01:26 INFO BlockManagerInfo: Removed broadcast_22_piece0 on spark-master:32891 in memory (size: 33.9 KiB, free: 434.4 MiB)
26/02/14 14:01:26 INFO BlockManagerInfo: Removed broadcast_22_piece0 on 172.18.0.3:35307 in memory (size: 33.9 KiB, free: 434.4 MiB)
26/02/14 14:01:26 INFO MemoryStore: Block broadcast_24_piece0 stored as bytes in memory (estimated size 33.9 KiB, free 434.1 MiB)
26/02/14 14:01:26 INFO BlockManagerInfo: Added broadcast_24_piece0 in memory on spark-master:32891 (size: 33.9 KiB, free: 434.4 MiB)
26/02/14 14:01:26 INFO SparkContext: Created broadcast 24 from count at NativeMethodAccessorImpl.java:0
26/02/14 14:01:26 INFO FileSourceScanExec: Planning scan with bin packing, max size: 4194304 bytes, open cost is considered as scanning 4194304 bytes
26/02/14 14:01:26 INFO BlockManagerInfo: Removed broadcast_23_piece0 on spark-master:32891 in memory (size: 7.7 KiB, free: 434.4 MiB)
26/02/14 14:01:26 INFO DAGScheduler: Registering RDD 61 (count at NativeMethodAccessorImpl.java:0) as input to shuffle 2
26/02/14 14:01:26 INFO DAGScheduler: Got map stage job 13 (count at NativeMethodAccessorImpl.java:0) with 1 output partitions
26/02/14 14:01:26 INFO BlockManagerInfo: Removed broadcast_23_piece0 on 172.18.0.3:35307 in memory (size: 7.7 KiB, free: 434.4 MiB)
26/02/14 14:01:26 INFO DAGScheduler: Final stage: ShuffleMapStage 15 (count at NativeMethodAccessorImpl.java:0)
26/02/14 14:01:26 INFO DAGScheduler: Parents of final stage: List()
```

```
26/02/14 14:01:32 INFO TaskSchedulerImpl: Killing all running tasks in stage 17: Stage finished
26/02/14 14:01:32 INFO DAGScheduler: Job 14 finished: count at NativeMethodAccessorImpl.java:0, took 0.254305 s
Total Rows: 500
```

```
26/02/14 14:01:33 INFO DAGScheduler: ResultStage 20 (showString at NativeMethodAccessorImpl.java:0) finished in 0.158
26/02/14 14:01:33 INFO DAGScheduler: Job 16 is finished. Cancelling potential speculative or zombie tasks for this job
26/02/14 14:01:33 INFO TaskSchedulerImpl: Killing all running tasks in stage 20: Stage finished
26/02/14 14:01:33 INFO DAGScheduler: Job 16 finished: showString at NativeMethodAccessorImpl.java:0, took 0.169073 s
+----------+------------+--------+-----+
|product_id|product_name|category|price|
+----------+------------+--------+-----+
|         0|           0|       0|    0|
+----------+------------+--------+-----+
```

```
26/02/14 14:01:33 INFO DAGScheduler: Job 17 is finished. Cancelling potential speculative or zombie tasks for this job
26/02/14 14:01:33 INFO TaskSchedulerImpl: Killing all running tasks in stage 21: Stage finished
26/02/14 14:01:33 INFO DAGScheduler: Job 17 finished: showString at NativeMethodAccessorImpl.java:0, took 0.349851 s
+----------+------------+---------+---------+
|product_id|product_name| category|    price|
+----------+------------+---------+---------+
|         1|   Product_1|utilities|1607261.0|
|         2|   Product_2|furniture|1246607.0|
|         3|   Product_3|groceries|1986808.0|
|         4|   Product_4|furniture| 988021.0|
|         5|   Product_5|furniture|1187756.0|
+----------+------------+---------+---------+
only showing top 5 rows
```

```
===== DATA PROFILING: ORDERS =====
26/02/14 14:01:34 INFO FileSourceStrategy: Pushed Filters:
26/02/14 14:01:34 INFO FileSourceStrategy: Post-Scan Filters:
26/02/14 14:01:34 INFO FileSourceStrategy: Output Data Schema: struct<>
26/02/14 14:01:34 INFO MemoryStore: Block broadcast_32 stored as values in memory (estimated size 199.2 KiB, free 434.0 MiB)
26/02/14 14:01:34 INFO BlockManagerInfo: Removed broadcast_31_piece0 on spark-master:32891 in memory (size: 7.3 KiB, free: 434.4 MiB)
26/02/14 14:01:34 INFO BlockManagerInfo: Removed broadcast_31_piece0 on 172.18.0.3:35307 in memory (size: 7.3 KiB, free: 434.4 MiB)
26/02/14 14:01:34 INFO BlockManagerInfo: Removed broadcast_30_piece0 on spark-master:32891 in memory (size: 33.9 KiB, free: 434.4 MiB)
26/02/14 14:01:34 INFO MemoryStore: Block broadcast_32_piece0 stored as bytes in memory (estimated size 33.9 KiB, free 434.2 MiB)
26/02/14 14:01:34 INFO BlockManagerInfo: Added broadcast_32_piece0 in memory on spark-master:32891 (size: 33.9 KiB, free: 434.4 MiB)
26/02/14 14:01:34 INFO BlockManagerInfo: Removed broadcast_30_piece0 on 172.18.0.3:35307 in memory (size: 33.9 KiB, free: 434.4 MiB)
26/02/14 14:01:34 INFO SparkContext: Created broadcast 32 from count at NativeMethodAccessorImpl.java:0
26/02/14 14:01:34 INFO FileSourceScanExec: Planning scan with bin packing, max size: 4194304 bytes, open cost is considered as scanning 4194304 bytes.
26/02/14 14:01:34 INFO DAGScheduler: Registering RDD 79 (count at NativeMethodAccessorImpl.java:0) as input to shuffle 4
26/02/14 14:01:34 INFO DAGScheduler: Got map stage job 18 (count at NativeMethodAccessorImpl.java:0) with 1 output partitions
26/02/14 14:01:34 INFO DAGScheduler: Final stage: ShuffleMapStage 22 (count at NativeMethodAccessorImpl.java:0)
26/02/14 14:01:34 INFO DAGScheduler: Parents of final stage: List()
```

```
26/02/14 14:01:34 INFO TaskSchedulerImpl: Killing all running tasks in stage 24: Stage finished
26/02/14 14:01:34 INFO DAGScheduler: Job 19 finished: count at NativeMethodAccessorImpl.java:0, took 0.154029 s
Total Rows: 3000
```

```
26/02/14 14:01:35 INFO TaskSchedulerImpl: Killing all running tasks in stage 27: Stage finished
26/02/14 14:01:35 INFO DAGScheduler: Job 21 finished: showString at NativeMethodAccessorImpl.java:0, took 0.086711 s
+--------+-------+----------+------------+
|order_id|user_id|order_date|total_amount|
+--------+-------+----------+------------+
|       0|      0|         0|           0|
+--------+-------+----------+------------+
```

```
26/02/14 14:01:36 INFO DAGScheduler: ResultStage 28 (showString at NativeMethodAccessorImpl.java:0) finished in 0.166 s
26/02/14 14:01:36 INFO DAGScheduler: Job 22 is finished. Cancelling potential speculative or zombie tasks for this job
26/02/14 14:01:36 INFO TaskSchedulerImpl: Killing all running tasks in stage 28: Stage finished
26/02/14 14:01:36 INFO DAGScheduler: Job 22 finished: showString at NativeMethodAccessorImpl.java:0, took 0.179766 s
+--------+-------+----------+------------+
|order_id|user_id|order_date|total_amount|
+--------+-------+----------+------------+
|    5001|    375|2023-04-23|   2115104.0|
|    5002|    201|2023-03-12|   2018053.0|
|    5003|    144|2023-01-30|   1143270.0|
|    5004|    543|2023-05-15|   2531019.0|
|    5005|    405|2023-03-08|    875126.0|
+--------+-------+----------+------------+
only showing top 5 rows
```

```
===== DATA PROFILING: ORDER_ITEMS =====
26/02/14 14:01:36 INFO FileSourceStrategy: Pushed Filters:
26/02/14 14:01:36 INFO FileSourceStrategy: Post-Scan Filters:
26/02/14 14:01:36 INFO FileSourceStrategy: Output Data Schema: struct<>
26/02/14 14:01:36 INFO MemoryStore: Block broadcast_40 stored as values in memory (estimated size 199.2 KiB, free 434.0 MiB)
26/02/14 14:01:36 INFO MemoryStore: Block broadcast_40_piece0 stored as bytes in memory (estimated size 33.9 KiB, free 433.9 MiB)
26/02/14 14:01:36 INFO BlockManagerInfo: Added broadcast_40_piece0 in memory on spark-master:32891 (size: 33.9 KiB, free: 434.3 MiB)
26/02/14 14:01:36 INFO SparkContext: Created broadcast 40 from count at NativeMethodAccessorImpl.java:0
26/02/14 14:01:36 INFO FileSourceScanExec: Planning scan with bin packing, max size: 4194304 bytes, open cost is considered as scanning 4194304 bytes.
26/02/14 14:01:36 INFO DAGScheduler: Registering RDD 97 (count at NativeMethodAccessorImpl.java:0) as input to shuffle 6
26/02/14 14:01:36 INFO DAGScheduler: Got map stage job 23 (count at NativeMethodAccessorImpl.java:0) with 1 output partitions
26/02/14 14:01:36 INFO DAGScheduler: Final stage: ShuffleMapStage 29 (count at NativeMethodAccessorImpl.java:0)
26/02/14 14:01:36 INFO DAGScheduler: Parents of final stage: List()
```

```
26/02/14 14:01:36 INFO DAGScheduler: ResultStage 31 (count at NativeMethodAccessorImpl.java:0) finished in 0.180 s
26/02/14 14:01:36 INFO DAGScheduler: Job 24 is finished. Cancelling potential speculative or zombie tasks for this job
26/02/14 14:01:36 INFO TaskSchedulerImpl: Killing all running tasks in stage 31: Stage finished
26/02/14 14:01:36 INFO DAGScheduler: Job 24 finished: count at NativeMethodAccessorImpl.java:0, took 0.190952 s
Total Rows: 3000
```

```
26/02/14 14:01:37 INFO DAGScheduler: Job 26 is finished. Cancelling potential speculative or zombie tasks for this job
26/02/14 14:01:37 INFO TaskSchedulerImpl: Killing all running tasks in stage 34: Stage finished
26/02/14 14:01:37 INFO DAGScheduler: Job 26 finished: showString at NativeMethodAccessorImpl.java:0, took 0.141462 s
+--------+----------+--------+-----+
|order_id|product_id|quantity|price|
+--------+----------+--------+-----+
|       0|         0|       0|    0|
+--------+----------+--------+-----+
```

```
26/02/14 14:01:37 INFO TaskSchedulerImpl: Removed TaskSet 35.0, whose tasks have all completed, from pool
26/02/14 14:01:37 INFO DAGScheduler: ResultStage 35 (showString at NativeMethodAccessorImpl.java:0) finished in 0.136 s
26/02/14 14:01:37 INFO DAGScheduler: Job 27 is finished. Cancelling potential speculative or zombie tasks for this job
26/02/14 14:01:37 INFO TaskSchedulerImpl: Killing all running tasks in stage 35: Stage finished
26/02/14 14:01:37 INFO DAGScheduler: Job 27 finished: showString at NativeMethodAccessorImpl.java:0, took 0.139961 s
+--------+----------+--------+---------+
|order_id|product_id|quantity|    price|
+--------+----------+--------+---------+
|    5001|       483|       1| 911307.0|
|    5002|       441|       3|1028008.0|
|    5003|       325|       4|1429849.0|
|    5004|       105|       3|  63923.0|
|    5005|       406|       3|1682060.0|
+--------+----------+--------+---------+
only showing top 5 rows
```

Perintah 3:

Buat “anomaly summary” awal:

1. jumlah orders dengan total_amount <= 0,
2. jumlah order_items dengan quantity <= 0 atau price <= 0,
3. jumlah duplikasi key utama (order_id di orders, product_id di products, user_id di users).

```
print("\n===== ANOMALY SUMMARY =====")

# Orders dengan total_amount <= 0
invalid_orders = orders_df.filter(col("total_amount") <= 0).count()
print(f"Jumlah orders dengan total_amount <= 0: {invalid_orders}")

# Order_items dengan quantity <= 0 atau price <= 0
invalid_order_items = order_items_df.filter(
    (col("quantity") <= 0) | (col("price") <= 0)
).count()
print(f"Jumlah order_items dengan quantity <= 0 atau price <= 0: {invalid_order_items}")

# Duplikasi primary key
# a) Duplicate order_id
duplicate_orders = orders_df.groupBy("order_id") \
    .count() \
    .filter(col("count") > 1) \
    .count()
print(f"Jumlah duplikasi order_id di orders: {duplicate_orders}")
# b) Duplicate product_id
duplicate_products = products_df.groupBy("product_id") \
    .count() \
    .filter(col("count") > 1) \
    .count()
print(f"Jumlah duplikasi product_id di products: {duplicate_products}")
# c) Duplicate user_id
```

```python
duplicate_users = users_df.groupBy("user_id") \
    .count() \
    .filter(col("count") > 1) \
    .count()
print(f"Jumlah duplikasi user_id di users: {duplicate_users}")
spark.stop()
```

```
===== ANOMALY SUMMARY =====
26/02/14 14:28:34 INFO FileSourceStrategy: Pushed Filters: IsNotNull(total_amount),LessThanOrEqual(total_amount,0)
26/02/14 14:28:34 INFO FileSourceStrategy: Post-Scan Filters: isnotnull(total_amount#70),(total_amount#70 <= 0)
26/02/14 14:28:34 INFO FileSourceStrategy: Output Data Schema: struct<total_amount: int>
26/02/14 14:28:34 INFO CodeGenerator: Code generated in 18.994929 ms
```

```
26/02/14 14:28:35 INFO DAGScheduler: Job 9 finished: count at NativeMethodAccessorImpl.java:0, took 0.413891 s
Jumlah orders dengan total_amount <= 0: 0
26/02/14 14:28:35 INFO FileSourceStrategy: Pushed Filters: Or(LessThanOrEqual(quantity,0),LessThanOrEqual(price,0))
```

```
26/02/14 14:28:36 INFO TaskSchedulerImpl: Killing all running tasks in stage 13: Stage finished
26/02/14 14:28:36 INFO DAGScheduler: Job 11 finished: count at NativeMethodAccessorImpl.java:0, took 0.094756 s
Jumlah order_items dengan quantity <= 0 atau price <= 0: 0
26/02/14 14:28:36 INFO FileSourceStrategy: Pushed Filters:
```

```
26/02/14 14:28:40 INFO BlockManagerInfo: Removed broadcast_24_piece0 on 172.18.0.3:38253 in memory (size: 18.7 KiB, free: 434.4 MiB)
Jumlah duplikasi order_id di orders: 0
```

```
26/02/14 14:28:42 INFO BlockManagerInfo: Removed broadcast_25_piece0 on spark-master:39469 in memory (size: 5.5 KiB, free: 434.3 MiB)
Jumlah duplikasi product_id di products: 0
26/02/14 14:28:42 INFO BlockManagerInfo: Removed broadcast_27_piece0 on 172.18.0.3:38253 in memory (size: 16.9 KiB, free: 434.4 MiB)
```

```
26/02/14 14:28:44 INFO TaskSchedulerImpl: Killing all running tasks in stage 31: Stage finished
26/02/14 14:28:44 INFO DAGScheduler: Job 20 finished: count at NativeMethodAccessorImpl.java:0, took 0.086849 s
Jumlah duplikasi user_id di users: 0
26/02/14 14:28:44 INFO SparkUI: Stopped Spark web UI at http://spark-master:4040
```

SOAL 2 — Cleaning & Standardization

Perintah 1:

Bersihkan data:

5. drop baris orders yang order_id atau user_id null,
6. drop baris order_items yang order_id atau product_id null,
7. drop baris products yang product_id null,
8. drop baris users yang user_id null.

```
print("\n===== CLEANING & STANDARDIZATION =====")

# Simpan jumlah awal
initial_users = users_df.count()
initial_products = products_df.count()
initial_orders = orders_df.count()
initial_order_items = order_items_df.count()

# Drop orders jika order_id atau user_id null
orders_clean = orders_df.dropna(subset=["order_id", "user_id"])

# Drop order_items jika order_id atau product_id null
order_items_clean = order_items_df.dropna(subset=["order_id",
"product_id"])

#  Drop products jika product_id null
products_clean = products_df.dropna(subset=["product_id"])

#  Drop users jika user_id null
users_clean = users_df.dropna(subset=["user_id"])

# Hitung setelah cleaning
final_users = users_clean.count()
final_products = products_clean.count()
final_orders = orders_clean.count()
final_order_items = order_items_clean.count()

# Buat summary table supaya mudah di screenshot
summary_data = [
    ("users", initial_users, final_users, initial_users - final_users),
    ("products", initial_products, final_products, initial_products -
final_products),
    ("orders", initial_orders, final_orders, initial_orders -
final_orders),
    ("order_items", initial_order_items, final_order_items,
initial_order_items - final_order_items),
]
```

```
summary_df = spark.createDataFrame(
    summary_data,
    ["dataset", "rows_before", "rows_after", "rows_dropped"]
)

summary_df.show(truncate=False)
```

```
+-----------+-----------+----------+------------+
|dataset    |rows_before|rows_after|rows_dropped|
+-----------+-----------+----------+------------+
|users      |500        |500       |0           |
|products   |500        |500       |0           |
|orders     |3000       |3000      |0           |
|order_items|3000       |3000      |0           |
+-----------+-----------+----------+------------+
```

Perintah 2:

Standardisasi kolom teks:

3. city di users → lowercase + trim,
4. category di products → lowercase + trim.

```
print("\n===== STANDARDIZATION =====")
# Standardisasi users.city
users_std = users_clean.withColumn(
    "city",
    trim(lower(col("city")))
)
# Standardisasi products.category
products_std = products_clean.withColumn(
    "category",
    trim(lower(col("category")))
)
# Untuk dataset lain tetap pakai hasil cleaning
orders_std = orders_clean
order_items_std = order_items_clean
# Buat ringkasan hasil standardisasi untuk screenshot
print("\nSample USERS after standardization:")
users_std.select("user_id", "city").show(5, truncate=False)
print("\nSample PRODUCTS after standardization:")
products_std.select("product_id", "category").show(5, truncate=False)
```

```
26/02/14 15:05:31 INFO CodeGenerator: Code generated in 48.943388 ms
+-------+--------+
|user_id|city    |
+-------+--------+
|101    |depok   |
|102    |medan   |
|103    |depok   |
|104    |surabaya|
|105    |depok   |
+-------+--------+
only showing top 5 rows
```

```
26/02/14 15:05:32 INFO TaskSchedulerImpl: Killing all running tasks in stage 9: Stage finished
26/02/14 15:05:32 INFO DAGScheduler: Job 9 finished: showString at NativeMethodAccessorImpl.java:0, took 1.150201 s
+----------+---------+
|product_id|category |
+----------+---------+
|1         |utilities|
|2         |furniture|
|3         |groceries|
|4         |furniture|
|5         |furniture|
+----------+---------+
only showing top 5 rows
```

Perintah 3:

Deduplication berbasis key:

5. orders dedupe by order_id,
6. order_items dedupe by kombinasi (order_id,product_id),
7. products dedupe by product_id,

users dedupe by user_id. Tunjukkan jumlah baris sebelum vs sesudah dedupe

```python
print("\n===== DEDUPLICATION SUMMARY =====")

# Simpan jumlah sebelum dedupe
before_users = users_std.count()
before_products = products_std.count()
before_orders = orders_std.count()
before_order_items = order_items_std.count()

# orders dedupe by order_id
orders_dedup = orders_std.dropDuplicates(["order_id"])

# order_items dedupe by (order_id, product_id)
order_items_dedup = order_items_std.dropDuplicates(["order_id", "product_id"])

# products dedupe by product_id
products_dedup = products_std.dropDuplicates(["product_id"])

# users dedupe by user_id
users_dedup = users_std.dropDuplicates(["user_id"])

# Hitung setelah dedupe
```

```python
after_users = users_dedup.count()
after_products = products_dedup.count()
after_orders = orders_dedup.count()
after_order_items = order_items_dedup.count()

# Buat summary table
dedupe_summary = [
    ("users", before_users, after_users, before_users - after_users),
    ("products", before_products, after_products, before_products -
after_products),
    ("orders", before_orders, after_orders, before_orders - after_orders),
    ("order_items", before_order_items, after_order_items, before_order_items -
after_order_items),
]

dedupe_summary_df = spark.createDataFrame(
    dedupe_summary,
    ["dataset", "rows_before", "rows_after", "rows_removed"]
)

dedupe_summary_df.show(truncate=False)
```

```
26/02/14 15:11:07 INFO DAGScheduler: Job 31 is finished. Cancelling potential speculative or zombie tasks for this job
26/02/14 15:11:07 INFO TaskSchedulerImpl: Killing all running tasks in stage 47: Stage finished
26/02/14 15:11:07 INFO DAGScheduler: Job 31 finished: showString at NativeMethodAccessorImpl.java:0, took 0.163851 s
26/02/14 15:11:08 INFO CodeGenerator: Code generated in 28.353643 ms
+-----------+-----------+----------+------------+
|dataset    |rows_before|rows_after|rows_removed|
+-----------+-----------+----------+------------+
|users      |500        |500       |0           |
|products   |500        |500       |0           |
|orders     |3000       |3000      |0           |
|order_items|3000       |3000      |0           |
+-----------+-----------+----------+------------+
```

SOAL 3 — Data Enrichment & Feature Engineering

Perintah 1:

Di order_items, buat kolom turunan:

2. total_item_value = quantity * price.

```
print("\n===== FEATURE ENGINEERING: total_item_value =====")

order_items_enriched = order_items_dedup.withColumn(
    "total_item_value",
    col("quantity") * col("price")
)

# Tampilkan contoh hasil
order_items_enriched.select(
    "order_id",
    "product_id",
    "quantity",
    "price",
    "total_item_value"
).show(5, truncate=False)
```

```
26/02/14 15:19:12 INFO DAGScheduler: Job 33 finished: showString at NativeMethodAccessorImpl.java:0, took 0.351034 s
26/02/14 15:19:12 INFO CodeGenerator: Code generated in 29.877659 ms
+--------+----------+--------+---------+----------------+
|order_id|product_id|quantity|price    |total_item_value|
+--------+----------+--------+---------+----------------+
|5035    |14        |2       |773315.0 |1546630.0       |
|5121    |315       |4       |730731.0 |2922924.0       |
|5131    |457       |3       |1925734.0|5777202.0       |
|5194    |159       |4       |1854927.0|7419708.0       |
|5502    |113       |3       |880359.0 |2641077.0       |
+--------+----------+--------+---------+----------------+
only showing top 5 rows

26/02/14 15:19:12 INFO SparkUI: Stopped Spark web UI at http://spark-master:4040
```

Perintah 2:

Di orders, buat kolom kategorisasi menggunakan when/otherwise:

2. order_value_flag dengan aturan:
   high_value jika total_amount > 500000, selain itu normal.

```
print("\n===== FEATURE ENGINEERING: order_value_flag =====")

orders_enriched = orders_dedup.withColumn(
    "order_value_flag",
    when(col("total_amount") > 500000, "high_value")
    .otherwise("normal")
)

# Tampilkan contoh hasil
orders_enriched.select(
    "order_id",
    "total_amount",
    "order_value_flag"
```

```
).show(10, truncate=False)
```

```
0.224667 s
26/02/14 15:22:53 INFO CodeGenerator: Code generated in 21.192488 ms
+--------+------------+----------------+
|order_id|total_amount|order_value_flag|
+--------+------------+----------------+
|5156    |1955430.0   |high_value      |
|5300    |1189694.0   |high_value      |
|5518    |780330.0    |high_value      |
|5803    |1087979.0   |high_value      |
|6336    |1824898.0   |high_value      |
|6357    |1481742.0   |high_value      |
|6397    |857566.0    |high_value      |
|6466    |2055286.0   |high_value      |
|6620    |1689745.0   |high_value      |
|6654    |1021675.0   |high_value      |
+--------+------------+----------------+
only showing top 10 rows
```

Perintah 3:

Buat kolom segmentasi user di users:

2. user_tenure_days = datediff(order_date_max_per_user, signup_date) (Hint: butuh join/agg kecil untuk order_date_max_per_user).
   tenure_segment = new (<=30 hari), mature (31–180), loyal (>180).

```
# Ambil order_date maksimal per user
order_date_max = orders_enriched.groupBy("user_id") \
    .agg(max("order_date").alias("order_date_max_per_user"))
users_joined = users_dedup.join(
    order_date_max,
    on="user_id",
    how="left"
)
users_with_tenure = users_joined.withColumn(
    "user_tenure_days",
    datediff(col("order_date_max_per_user"), col("signup_date"))
)

users_final = users_with_tenure.withColumn(
    "tenure_segment",
    when(col("user_tenure_days") <= 30, "new")
    .when((col("user_tenure_days") > 30) & (col("user_tenure_days") <= 180), "mature")
    .when(col("user_tenure_days") > 180, "loyal")
    .otherwise("no_order")
)
users_final.select(
    "user_id",
    "signup_date",
```

```
    "order_date_max_per_user",
    "user_tenure_days",
    "tenure_segment"
).show(10, truncate=False)

```

```
26/02/14 15:29:15 INFO TaskSchedulerImpl: Killing all running tasks in stage 62: Stage finished
26/02/14 15:29:15 INFO DAGScheduler: Job 40 finished: showString at NativeMethodAccessorImpl.java:0, took 0.232344 s
26/02/14 15:29:15 INFO CodeGenerator: Code generated in 13.667162 ms
+-------+-----------+----------------------+----------------+--------------+
|user_id|signup_date|order_date_max_per_user|user_tenure_days|tenure_segment|
+-------+-----------+----------------------+----------------+--------------+
|148    |2022-02-17 |2023-06-17            |485             |loyal         |
|463    |2022-12-29 |2023-05-28            |150             |mature        |
|471    |2023-01-06 |2023-05-08            |122             |mature        |
|496    |2023-01-31 |2023-03-17            |45              |mature        |
|243    |2022-05-23 |2023-06-13            |386             |loyal         |
|392    |2022-10-19 |2023-04-08            |171             |mature        |
|540    |2023-03-16 |2023-06-28            |104             |mature        |
|516    |2023-02-20 |2023-06-19            |119             |mature        |
|137    |2022-02-06 |2023-05-28            |476             |loyal         |
|251    |2022-05-31 |2023-06-10            |375             |loyal         |
+-------+-----------+----------------------+----------------+--------------+
only showing top 10 rows

26/02/14 15:29:15 INFO SparkUI: Stopped Spark web UI at http://spark-master:4040
26/02/14 15:29:15 INFO StandaloneSchedulerBackend: Shutting down all executors
```

SOAL 4 — Join & Aggregation (Business Metrics)

Perintah 1:

Bangun DataFrame “fact” hasil join:
orders ↔ order_items ↔ products ↔ users
dengan key:

4. orders ↔ order_items: order_id
5. order_items ↔ products: product_id
6. orders ↔ users: user_id

Perintah 2:

Hitung metrik per **category per day**:

4. daily_gmv_category = sum(total_item_value)
5. daily_orders_category = countDistinct(order_id)
6. daily_items_sold = sum(quantity)
   Urutkan per hari dan GMV terbesar.

Perintah 3:

Hitung metrik per **city**:

5. total GMV kota,
6. AOV kota (avg total_amount),
7. jumlah customer unik kota,
8. top 3 kategori kontribusi terbesar per kota (boleh pakai window rank).

```
9. # ==============================
10.#  BUILD FACT TABLE
11.# ==============================
12.fact_df = orders \
13.    .join(order_items, on="order_id", how="inner") \
14.    .join(products, on="product_id", how="inner") \
15.    .join(users, on="user_id", how="inner")
16.
17.# ==============================
18.#  DAILY METRICS PER CATEGORY
19.# ==============================
20.daily_category_metrics = fact_df.groupBy(
21.    "order_date",
22.    "category"
23.).agg(
24.    sum("total_item_value").alias("daily_gmv_category"),
25.    countDistinct("order_id").alias("daily_orders_category"),
26.    sum("quantity").alias("daily_items_sold")
27.).orderBy(
28.    "order_date",
29.    col("daily_gmv_category").desc()
30.)
31.
```

```python
print("\n===== DAILY METRICS PER CATEGORY =====")
daily_category_metrics.show(50, truncate=False)

# ==============================
#  CITY LEVEL METRICS
# ==============================
city_metrics = fact_df.groupBy("city").agg(
    sum("total_item_value").alias("total_gmv_city"),
    avg("total_amount").alias("aov_city"),
    countDistinct("user_id").alias("unique_customers")
).orderBy(col("total_gmv_city").desc())

print("\n===== CITY LEVEL METRICS =====")
city_metrics.show(truncate=False)

# ==============================
# TOP 3 CATEGORY PER CITY
# ==============================
city_category_gmv = fact_df.groupBy(
    "city",
    "category"
).agg(
    sum("total_item_value").alias("gmv_per_category")
)

window_spec = Window.partitionBy("city") \
    .orderBy(col("gmv_per_category").desc())

top3_category_city = city_category_gmv.withColumn(
    "rank",
    rank().over(window_spec)
).filter(col("rank") <= 3)

print("\n===== TOP 3 CATEGORY PER CITY =====")
top3_category_city.orderBy("city", "rank").show(truncate=False)
```

```
+--------+-----------+----------------+----+
|city    |category   |gmv_per_category|rank|
+--------+-----------+----------------+----+
|bandung |electronics|3.52823575E8    |1   |
|bandung |groceries  |2.35720733E8    |2   |
|bandung |fashion    |2.35655086E8    |3   |
|bekasi  |fashion    |2.80857491E8    |1   |
|bekasi  |electronics|2.62640663E8    |2   |
|bekasi  |utilities  |2.61228351E8    |3   |
|depok   |electronics|3.03989612E8    |1   |
|depok   |fashion    |2.9340691E8     |2   |
|depok   |utilities  |2.90314341E8    |3   |
|jakarta |utilities  |3.54904568E8    |1   |
|jakarta |electronics|3.08101948E8    |2   |
|jakarta |furniture  |2.70057049E8    |3   |
|medan   |utilities  |2.68076553E8    |1   |
|medan   |fashion    |2.40031223E8    |2   |
|medan   |electronics|2.31796919E8    |3   |
|surabaya|electronics|2.86331382E8    |1   |
|surabaya|fashion    |2.56157569E8    |2   |
|surabaya|groceries  |2.56068031E8    |3   |
+--------+-----------+----------------+----+
```

SOAL 5 — Window Analytics & Insight Final

Perintah 1:

Gunakan window function per user untuk membuat kolom:

5. user_total_spending (sum total_amount per user),
6. user_total_orders (count order per user),
7. user_avg_order_value (avg total_amount per user),
8. user_rank global berdasarkan user_total_spending (dense_rank).

Perintah 2:

Buat analisis **cumulative GMV harian**:

3. gmv_daily,
4. cumulative_gmv (sum gmv_daily over order by date).

Perintah 3:

Ambil **Top 10 customers** (rank 1–10) dan tampilkan:
user_id, name, city, user_total_orders, user_total_spending, user_avg_order_value.
Tambahkan interpretasi singkat 4–6 kalimat: pola apa yang kamu lihat? (misal kota dominan, AOV tinggi tapi order sedikit, dll).

```
# ==============================
# WINDOW ANALYTICS PER USER
# ==============================
print("\n===== WINDOW ANALYTICS PER USER =====")

user_metrics = fact_df.groupBy(
    "user_id",
    "name",
```

```
    "city"
).agg(
    sum("total_amount").alias("user_total_spending"),
    countDistinct("order_id").alias("user_total_orders"),
    avg("total_amount").alias("user_avg_order_value")
)

rank_window = Window.orderBy(col("user_total_spending").desc())

user_metrics = user_metrics.withColumn(
    "user_rank",
    dense_rank().over(rank_window)
)

user_metrics.orderBy("user_rank").show(20, truncate=False)

# ==============================
# CUMULATIVE GMV DAILY
# ==============================
print("\n===== CUMULATIVE GMV DAILY =====")

gmv_daily = fact_df.groupBy("order_date") \
    .agg(sum("total_item_value").alias("gmv_daily")) \
    .orderBy("order_date")

cumulative_window = Window.orderBy("order_date") \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

gmv_daily = gmv_daily.withColumn(
    "cumulative_gmv",
    sum("gmv_daily").over(cumulative_window)
)

gmv_daily.show(50, truncate=False)

# ==============================
# TOP 10 CUSTOMERS
# ==============================
print("\n===== TOP 10 CUSTOMERS =====")

top10_customers = user_metrics \
    .filter(col("user_rank") <= 10) \
    .orderBy("user_rank")

top10_customers.select(
```

```
    "user_id",
    "name",
    "city",
    "user_total_orders",
    "user_total_spending",
    "user_avg_order_value"
).show(truncate=False)
```

```
26/02/15 03:59:08 INFO CodeGenerator: Code generated in 24.701479 ms
+-------+--------+-------+-----------------+-------------------+--------------------+
|user_id|name    |city   |user_total_orders|user_total_spending|user_avg_order_value|
+-------+--------+-------+-----------------+-------------------+--------------------+
|112    |User_112|medan  |14               |2.5933294E7        |1852378.142857143   |
|532    |User_532|bekasi |10               |2.1081558E7        |2108155.8           |
|454    |User_454|bandung |11               |2.0453741E7        |1859431.0           |
|495    |User_495|depok  |10               |1.9932321E7        |1993232.1           |
|393    |User_393|bandung |10               |1.9454982E7        |1945498.2           |
|217    |User_217|depok  |12               |1.8948369E7        |1579030.75          |
|563    |User_563|jakarta |10               |1.884323E7         |1884323.0           |
|545    |User_545|surabaya|10               |1.872886E7         |1872886.0           |
|402    |User_402|bandung |11               |1.8691645E7        |1699240.4545454546  |
|416    |User_416|jakarta |9                |1.8346701E7        |2038522.3333333333  |
+-------+--------+-------+-----------------+-------------------+--------------------+

26/02/15 03:59:08 INFO SparkUI: Stopped Spark web UI at http://spark-master:4040
26/02/15 03:59:08 INFO StandaloneSchedulerBackend: Shutting down all executors
```